

```
"""
```

```
Shadow Node Theory – Verificacion Cuantitativa  
Script de analisis de ley de potencia para los cuatro casos historicos  
Fractal Core Research | Tlaxcala, Mexico | 2026
```

```
Uso:
```

```
python shadow_node_verification.py
```

```
Dependencias:
```

```
pip install numpy scipy pandas matplotlib
```

```
Salida:
```

```
- shadow_node_resultados.csv    : exponentes y metricas por caso  
- shadow_node_graficas.png     : visualizacion de los cuatro casos
```

```
"""
```

```
import numpy as np  
import pandas as pd  
from scipy.optimize import curve_fit  
from scipy.stats import pearsonr, spearmanr  
import warnings  
warnings.filterwarnings('ignore')
```

```
# =====  
# DATOS HISTORICOS CONSOLIDADOS  
# Unidad: ratio = poblacion_nodo_dominante / poblacion_nodo_sombra  
# Un ratio > 1 significa que el nodo dominante ya supero al nodo sombra  
# =====
```

```
CASOS = {
```

```
# -----  
# CASO 1: BRUJAS -> AMBERES (1300-1589)  
# Mecanismo: colapso de infraestructura fisica (sedimentacion Canal Zwin)  
# Fuentes: Nicholas 1992, Van der Wee 1963, Gelderblom 2013  
# Proxy: poblacion total estimada (miles de habitantes)  
# -----  
"Brujas_Amberes": {  
    "descripcion": "Flamenco: Brujas → Amberes (1300-1589)",  
    "mecanismo": "Colapso infraestructura fisica (Canal Zwin)",  
    "trigger_año": 1490,  
    "variable": "Poblacion estimada (miles hab.)",  
    "años": [1300, 1375, 1450, 1500, 1520, 1560, 1589],  
    "nodo_sombra": [46, 42, 38, 30, 22, 15, 8], # Brujas
```

```

    "nodo_dominante": [5, 10, 18, 45, 65, 105, 42], # Amberes
    # Nota: caída de Amberes en 1589 por ocupacion española
    # Para el analisis de satelizacion usamos hasta 1560
    "notas": "Brujas pico siglo XIV. Amberes colapsa 1585-1589 por ocupacion
},

# -----
# CASO 2: TOLEDO -> MADRID (1528-1787)
# Mecanismo: decreto politico puro (traslado de corte 1561)
# Fuentes: Ringrose 1973, INE España
# Proxy: poblacion total (habitantes)
# -----
"Toledo_Madrid": {
    "descripcion": "Castellano: Toledo → Madrid (1528-1787)",
    "mecanismo": "Decreto politico puro (traslado de corte)",
    "trigger_año": 1561,
    "variable": "Poblacion (hab.)",
    "años": [1528, 1561, 1600, 1661, 1750, 1787],
    "nodo_sombra": [31930, 56270, 40000, 25000, 18000, 15000], # To
    "nodo_dominante": [5000, 15000, 55000, 120000, 140000, 156672], # Ma
    # Nota: Toledo 1600-1750 estimados por interpolacion de Ringrose
    # Toledo 1787: estimacion basada en tendencia de declive documentada
    "notas": "Toledo 1600-1787 estimados por interpolacion. Madrid desde dato
},

# -----
# CASO 3: PORTUGAL vs NUCLEO NOROESTE EUROPEO (1500-1800)
# CORRECCION METODOLOGICA: Portugal y España cayeron juntos en la trampa
# malthusiana. El nodo dominante correcto es Inglaterra/Paises Bajos,
# no España. La satelizacion fue respecto al sistema global, no al
# nodo inmediato.
# Fuentes: Costa, Palma y Reis 2015; Allen 2001; Broadberry et al. 2015
# Proxy: salario real cualificado (unidades canasta Estrasburgo)
# -----
"Portugal_NW_Europa": {
    "descripcion": "Iberico: Portugal vs Inglaterra/PaisesBajos (1500-1800)",
    "mecanismo": "Satelizacion sistémica: trampa malthusiana + Union Iberica
    "trigger_año": 1580,
    "variable": "Salario real cualificado (unidades canasta Estrasburgo)",
    "años": [1500, 1600, 1700, 1750, 1800],
    "nodo_sombra": [8.9, 5.9, 5.1, 6.9, 4.5], # Portugal
    "nodo_dominante": [10.5, 11.0, 11.5, 12.0, 10.5], # Inglaterra/Holanda (
    # Nota: salarios NW Europa de Allen 2001. Inglaterra resistio presion
    # malthusiana; Portugal no. Brecha se amplifica post-1580.
    # Fuente: Allen R.C. (2001) The Great Divergence in European Wages.
    # Explorations in Economic History 38(4), 411-447.
    "notas": "Comparacion correcta: Portugal vs nucleo NW Europa, no vs Españ

```

```

},

# -----
# CASO 4: TLAXCALA -> PUEBLA (1535-2022) SERIE LARGA
# Mecanismo: ventaja acumulada gradual (Real Cedula 1535 + bypass 1873)
# Fuentes precoloniales: Maddison Project / Arroyo Abad & van Zanden 2012
# Fuentes modernas: INEGI, CONEVAL, Secretaria de Economia
# Proxy: PIB per capita en USD internacionales 2011 (Maddison)
# NOTA: datos 1535-1820 son estimaciones para America española
# basadas en salarios reales de Mexico City (Arroyo Abad et al. 2012)
# calibrados al nivel Maddison 1820. Datos 1993-2022 de INEGI indexados.
# -----
"Tlaxcala_Puebla": {
    "descripcion": "Mesoamericano: Tlaxcala → Puebla (1535-2022)",
    "mecanismo": "Ventaja acumulada gradual (colonial + infraestructura)",
    "trigger_año": 1535,
    "variable": "PIB per capita relativo (base Tlaxcala=100 en 1535)",
    "años": [1535, 1600, 1650, 1700, 1750, 1800, 1820, 1870, 1910, 1940, 1960
    # Tlaxcala: region agricola periferica bajo dominio colonial
    # Indice basado en que Tlaxcala mantiene nivel subsistencia durante
    # colonia mientras Puebla acumula ventaja institucional
    "nodo_sombra": [100, 95, 92, 90, 88, 85, 83, 80, 78, 82, 88,
    # Puebla: ciudad colonial con obispado, Universidad (1587), manufactura t
    # Ventaja inicial c.1535: 10-15% por designacion como ciudad principal
    # Ventaja acumulada a 1820: estimada en 40-50% por Maddison latinoamerica
    # Ventaja confirmada 1993: 48.8% (INEGI)
    "nodo_dominante": [112, 120, 128, 135, 140, 142, 140, 145, 148, 150, 155,
    "notas": "Serie larga 1535-2022. Pre-1993: estimaciones basadas en Arroyo
}
}

# =====
# FUNCIONES DE ANALISIS
# =====

def ley_potencia(t, a, b):
    """
    Modelo de ley de potencia:  $f(t) = a * t^b$ 
    Para el ratio dominante/sombra en funcion del tiempo desde trigger
    """
    return a * np.power(t, b)

def calcular_ratio_serie(años, nodo_sombra, nodo_dominante, trigger_año):
    """
    Calcula la serie de ratios y el tiempo desde el trigger.
    Ratio = nodo_dominante / nodo_sombra
    Un ratio creciente indica satelizacion en progreso.

```

```

"""
ratios = []
tiempos = []
for i, año in enumerate(años):
    if nodo_sombra[i] > 0:
        ratio = nodo_dominante[i] / nodo_sombra[i]
        tiempo = max(año - trigger_año, 1) # evitar log(0)
        ratios.append(ratio)
        tiempos.append(tiempo)
return np.array(tiempos), np.array(ratios)

def ajustar_ley_potencia(tiempos, ratios, nombre_caso):
    """
    Ajusta una ley de potencia  $f(t) = a * t^b$  a los datos.
    Retorna parametros, R2, y diagnosticos.
    """
    resultado = {
        "caso": nombre_caso,
        "n_puntos": len(tiempos),
        "a": None, "b": None,
        "r2": None,
        "r_pearson": None,
        "p_value": None,
        "interpretacion": ""
    }

    if len(tiempos) < 3:
        resultado["interpretacion"] = "Insuficientes puntos para ajuste"
        return resultado

    try:
        # Ajuste de ley de potencia en espacio logaritmico para mayor estabilidad
        log_t = np.log(tiempos)
        log_r = np.log(ratios)

        # Regresion lineal en log-log
        coef = np.polyfit(log_t, log_r, 1)
        b = coef[0]
        a = np.exp(coef[1])

        # R2 en escala original
        ratios_pred = ley_potencia(tiempos, a, b)
        ss_res = np.sum((ratios - ratios_pred) ** 2)
        ss_tot = np.sum((ratios - np.mean(ratios)) ** 2)
        r2 = 1 - (ss_res / ss_tot) if ss_tot > 0 else 0

        # Correlacion de Pearson en escala log

```

```

r_val, p_val = pearsonr(log_t, log_r)

resultado.update({
    "a": round(a, 4),
    "b": round(b, 4),
    "r2": round(r2, 4),
    "r_pearson": round(r_val, 4),
    "p_value": round(p_val, 6)
})

# Interpretacion del exponente b
if b > 0.5:
    resultado["interpretacion"] = "Satelizacion acelerada (divergencia su
elif b > 0.1:
    resultado["interpretacion"] = "Satelizacion gradual (divergencia sub-
elif b > -0.1:
    resultado["interpretacion"] = "Sistema estabilizado (ratio aproximada
else:
    resultado["interpretacion"] = "Convergencia (nodo sombra recuperando

except Exception as e:
    resultado["interpretacion"] = f"Error en ajuste: {str(e)}"

return resultado

def calcular_umbral_activacion(años, nodo_sombra, nodo_dominante):
    """
    Identifica el año en que el nodo dominante supero el umbral del 10-15%
    de ventaja sobre el nodo sombra, segun predice la Shadow Node Theory.
    """
    for i, año in enumerate(años):
        if nodo_sombra[i] > 0:
            ventaja = (nodo_dominante[i] - nodo_sombra[i]) / nodo_sombra[i]
            if ventaja >= 0.10:
                return año, round(ventaja * 100, 1)
    return None, None

def calcular_velocidad_divergencia(años, nodo_sombra, nodo_dominante, trigger_año):
    """
    Calcula la velocidad de divergencia: cambio en el ratio por año
    en los primeros 50 años despues del trigger.
    """
    ratios_post = []
    años_post = []
    for i, año in enumerate(años):
        if año >= trigger_año and año <= trigger_año + 100:
            if nodo_sombra[i] > 0:

```

```

        ratios_post.append(nodo_dominante[i] / nodo_sombra[i])
        años_post.append(año)

    if len(años_post) >= 2:
        delta_ratio = ratios_post[-1] - ratios_post[0]
        delta_años = años_post[-1] - años_post[0]
        velocidad = delta_ratio / delta_años if delta_años > 0 else 0
        return round(velocidad, 6)
    return None

# =====
# EJECUCION PRINCIPAL
# =====

print("=" * 70)
print("SHADOW NODE THEORY – VERIFICACION CUANTITATIVA")
print("Fractal Core Research | Tlaxcala, Mexico | 2026")
print("=" * 70)

resultados_completos = []

for nombre, caso in CASOS.items():

    print(f"\n{'-' * 60}")
    print(f"CASO: {caso['descripcion']}")
    print(f"Mecanismo: {caso['mecanismo']}")
    print(f"Trigger: {caso['trigger_año']}")
    print(f"Variable: {caso['variable']}")

    años = np.array(caso["años"])
    sombra = np.array(caso["nodo_sombra"], dtype=float)
    dominante = np.array(caso["nodo_dominante"], dtype=float)
    trigger = caso["trigger_año"]

    # Serie de ratios
    tiempos, ratios = calcular_ratio_serie(
        caso["años"], caso["nodo_sombra"],
        caso["nodo_dominante"], trigger
    )

    print(f"\nSerie de ratios (dominante/sombra):")
    for i, año in enumerate(caso["años"]):
        if sombra[i] > 0:
            r = dominante[i] / sombra[i]
            t = año - trigger
            print(f"  {año} (t={t:+d}): ratio = {r:.3f}")

```

```

# Ajuste ley de potencia
ajuste = ajustar_ley_potencia(tiempos, ratios, nombre)
print(f"\nAjuste ley de potencia  $f(t) = a * t^b$ :")
print(f"  a = {ajuste['a']}")
print(f"  b = {ajuste['b']} ← exponente clave")
print(f"  R2 = {ajuste['r2']}")
print(f"  r Pearson (log-log) = {ajuste['r_pearson']}")
print(f"  p-value = {ajuste['p_value']}")
print(f"  Interpretacion: {ajuste['interpretacion']}")

# Umbral de activacion
año_umbral, ventaja_umbral = calcular_umbral_activacion(
    caso["años"], caso["nodo_sombra"], caso["nodo_dominante"]
)
if año_umbral:
    print(f"\nUmbral 10-15% superado: año {año_umbral} (ventaja: {ventaja_umb
else:
    print(f"\nUmbral 10-15%: no detectado en los datos disponibles")

# Velocidad de divergencia
velocidad = calcular_velocidad_divergencia(
    caso["años"], caso["nodo_sombra"],
    caso["nodo_dominante"], trigger
)
if velocidad is not None:
    print(f"Velocidad divergencia post-trigger: {velocidad:.4f} unidades de r

# Acumular para CSV
resultados_completos.append({
    "caso": nombre,
    "descripcion": caso["descripcion"],
    "mecanismo": caso["mecanismo"],
    "trigger_año": trigger,
    "n_puntos": ajuste["n_puntos"],
    "exponente_b": ajuste["b"],
    "coeficiente_a": ajuste["a"],
    "r2": ajuste["r2"],
    "r_pearson": ajuste["r_pearson"],
    "p_value": ajuste["p_value"],
    "año_umbral_10pct": año_umbral,
    "ventaja_en_umbral_pct": ventaja_umbral,
    "velocidad_divergencia_ratio_por_año": velocidad,
    "interpretacion": ajuste["interpretacion"],
    "notas": caso["notas"]
})

# =====

```

```

# TABLA COMPARATIVA FINAL
# =====

print(f"\n{'=' * 70}")
print("TABLA COMPARATIVA - LOS CUATRO CASOS")
print("=" * 70)

df = pd.DataFrame(resultados_completos)

cols_display = ["caso", "mecanismo", "exponente_b", "r2", "año_umbral_10pct", "ve
print(df[cols_display].to_string(index=False))

# =====
# ANALISIS DE CONVERGENCIA DE EXPONENTES
# =====

print(f"\n{'-' * 60}")
print("ANALISIS DE CONVERGENCIA DE EXPONENTES (hipotesis central)")
print("-" * 60)

exponentes = [r["exponente_b"] for r in resultados_completos if r["exponente_b"]
if len(exponentes) >= 3:
    media_b = np.mean(exponentes)
    std_b = np.std(exponentes)
    cv_b = std_b / media_b if media_b != 0 else float('inf')

    print(f"Exponentes por caso: {[round(b, 3) for b in exponentes]}")
    print(f"Media: {media_b:.4f}")
    print(f"Desviacion estandar: {std_b:.4f}")
    print(f"Coeficiente de variacion: {cv_b:.4f}")
    print()

    if cv_b < 0.30:
        print("RESULTADO: Exponentes convergentes (CV < 0.30)")
        print("Interpretacion: El mismo algoritmo opera en todos los casos.")
        print("Esto apoya la hipotesis central de la Shadow Node Theory.")
    elif cv_b < 0.60:
        print("RESULTADO: Exponentes moderadamente similares (0.30 < CV < 0.60)")
        print("Interpretacion: Patron comun con variacion por mecanismo de activa
        print("Investigacion adicional recomendada.")
    else:
        print("RESULTADO: Exponentes divergentes (CV > 0.60)")
        print("Interpretacion: Los casos no comparten el mismo patron matematico.
        print("La hipotesis de invarianza de escala requiere revision.")

# =====
# EXPORTAR RESULTADOS

```



```

# =====

df.to_csv("shadow_node_resultados.csv", index=False, encoding="utf-8-sig")
print(f"\nResultados exportados a: shadow_node_resultados.csv")

# =====
# INTENTAR GRAFICA (opcional, no falla si matplotlib no esta instalado)
# =====

try:
    import matplotlib.pyplot as plt
    import matplotlib.gridspec as gridspec

    fig = plt.figure(figsize=(16, 12))
    fig.suptitle("Shadow Node Theory – Divergencia entre Nodo Sombra y Nodo Dominante",
                 fontsize=14, fontweight='bold', y=0.98)

    gs = gridspec.GridSpec(2, 2, figure=fig, hspace=0.45, wspace=0.35)

    colores = ["#1B3A6B", "#2E6099", "#C0392B", "#27AE60"]
    titulos = ["Brujas → Amberes\n(Infraestructura)", "Toledo → Madrid\n(Decreto)",
               "Portugal → España\n(Vacío institucional)", "Tlaxcala → Puebla\n(Vacío institucional)"]

    for idx, (nombre, caso) in enumerate(CASOS.items()):
        ax = fig.add_subplot(gs[idx // 2, idx % 2])

        años = np.array(caso["años"])
        sombra = np.array(caso["nodo_sombra"], dtype=float)
        dominante = np.array(caso["nodo_dominante"], dtype=float)

        # Normalizar a 1.0 en el primer año para comparabilidad
        norm = sombra[0]
        sombra_norm = sombra / norm
        dominante_norm = dominante / norm

        ax.plot(años, sombra_norm, 'o-', color=colores[idx], linewidth=2,
                markersize=6, label="Nodo Sombra", alpha=0.9)
        ax.plot(años, dominante_norm, 's--', color=colores[idx], linewidth=2,
                markersize=6, label="Nodo Dominante", alpha=0.6)

        # Marcar trigger
        trigger = caso["trigger_año"]
        ax.axvline(x=trigger, color='red', linestyle=':', linewidth=1.5,
                  alpha=0.7, label=f"Trigger ({trigger})")

        # Exponente b del ajuste
        res = resultados_completos[idx]

```

```

    if res["exponente_b"] is not None:
        ax.text(0.05, 0.92, f"b = {res['exponente_b']}", transform=ax.transAx,
                fontsize=10, color=colores[idx], fontweight='bold',
                bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=

ax.set_title(titulos[idx], fontsize=11, fontweight='bold', color="#1B3A6B
ax.set_xlabel("Año", fontsize=9)
ax.set_ylabel(f"Indice (año inicial = 1.0)", fontsize=9)
ax.legend(fontsize=8, loc='upper left')
ax.grid(True, alpha=0.3)
ax.tick_params(labelsize=8)

plt.savefig("shadow_node_graficas.png", dpi=150, bbox_inches='tight',
            facecolor='white')
plt.close()
print("Grafica exportada a: shadow_node_graficas.png")

except ImportError:
    print("matplotlib no instalado. Instalar con: pip install matplotlib")
except Exception as e:
    print(f"Error generando grafica: {e}")

print(f"\n{'=' * 70}")
print("EJECUCION COMPLETADA")
print("Archivos generados:")
print("  - shadow_node_resultados.csv")
print("  - shadow_node_graficas.png  (si matplotlib disponible)")
print(f"{'=' * 70}")

```